



UFRJ



INSTITUTO DE
COMPUTAÇÃO
UFRJ

Programação de Computadores II

Laboratório - Alocação Dinâmica de Memória

Profa. Giseli Rabello Lopes

Exercício 1

- Escreva uma função **concatena** que tem o seguinte protótipo:

char *concatena(char *s1, char *s2);

A função deve, a partir de duas cadeias de caracteres (acessadas a partir dos ponteiros **s1** e **s2**), criar uma nova que é a concatenação da segunda cadeia ao final da primeira. Esta nova cadeia deve ser alocada dinamicamente pela função cujo retorno será um apontador para esta.

- Faça um programa principal para testar a função implementada.

Exercício 2

- Considere o programa principal a seguir e implemente as seguintes funções, necessárias para o seu adequado funcionamento:

```
int **aloca_linhas(int );  
int aloca_colunas (int **, int, int);  
void le_dados(int **, int, int );  
void imprime_matriz(int **, int, int );  
void troca_linhas (int **, int, int);
```

Exercício 2 (cont.)

- **aloca_linhas** deve alocar dinamicamente os ponteiros para as linhas da matriz
 - **aloca_colunas** deve alocar dinamicamente os elementos (colunas) de cada uma das linhas da matriz
 - **le_dados** deve solicitar ao usuário a entrada (a partir do teclado) de cada um dos elementos da matriz (elementos de cada linha da matriz)
 - **imprime_matriz** deve exibir (em tela) cada um dos elementos da matriz (conforme exemplo de saída)
 - **troca_linhas** deve realizar a troca dos elementos da matriz (alterando os ponteiros das linhas e não substituindo os elementos em si)
-

```
#include <stdio.h>
#include <stdlib.h>

int **aloca_linhas(int );
int aloca_colunas (int **, int, int);
void le_dados(int **, int, int );
void imprime_matriz(int **, int, int );
void troca_linhas (int **, int, int);

int main (void) {
    int **matriz;
    int lin, col;
    int linha1, linha2;

    scanf("%d",&lin); //número de linhas da matriz
    matriz = aloca_linhas(lin);
    if (matriz==NULL)
        return 1; //não foi possível alocar as linhas da matriz
    scanf("%d",&col); //número de colunas da matriz
    if ((aloca_colunas(matriz, lin, col))==1)
        return 1; //não foi possível alocar todas as colunas da matriz
    le_dados(matriz, lin, col); //ler os elementos da matriz
    imprime_matriz(matriz, lin, col); //matriz original
    scanf("%d",&linha1); //informar a primeira linha a ser trocada
    scanf("%d",&linha2); //informar a segunda linha a ser trocada
    troca_linhas(matriz, linha1, linha2);
    printf("\n");
    imprime_matriz(matriz, lin, col); //matriz após a troca das linhas
    return 0;
}
```

Exercício 2 (cont.)

- Exemplos de entrada e saídas:

Entrada 3 2 1 2 3 4 5 6 0 2 Saída [1] [2] [3] [4] [5] [6] [5] [6] [3] [4] [1] [2]	Entrada 4 3 1 2 3 4 5 6 7 8 9 10 11 12 2 3 Saída [1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [1] [2] [3] [4] [5] [6] [10] [11] [12] [7] [8] [9]	Entrada 4 2 5 7 4 9 1 0 8 6 1 2 Saída [5] [7] [4] [9] [1] [0] [8] [6] [5] [7] [1] [0] [4] [9] [8] [6]	Entrada 5 4 0 9 8 7 6 5 4 3 2 1 -1 -2 -3 -4 0 1 1 2 3 4 0 4 Saída [0] [9] [8] [7] [6] [5] [4] [3] [2] [1] [-1] [-2] [-3] [-4] [0] [1] [1] [2] [3] [4] [1] [2] [3] [4] [6] [5] [4] [3] [2] [1] [-1] [-2] [-3] [-4] [0] [1] [0] [9] [8] [7]
---	---	--	---

Exercício 3

- Considere o programa principal a seguir e implemente as seguintes funções, necessárias para o seu adequado funcionamento:

```
void * inicializa (Tfunc *, int);  
void * inclui_novos(Tfunc *, int, int);  
void imprime (Tfunc *, int, int);
```

Exercício 3 (cont.)

- **inicializa** deve alocar dinamicamente o vetor de estruturas e armazenar as informações de cada funcionário solicitadas ao usuário (a partir do teclado)
 - **inclui_novos** deve realocar dinamicamente o vetor de estruturas para armazenar as informações de mais usuários
 - **imprime** deve exibir (em tela) as informações dos funcionários no intervalo de posições indicado; para mostrar as informações de todos os funcionários, o intervalo deve ser 0 a **funcionarios+adicionais**
-

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct _func
{
    char nome[40];
    int ano_nasc;
    float renda;
} Tfunc ;

void * inicializa (Tfunc *, int);
void * inclui_novos(Tfunc *, int, int);
void imprime (Tfunc *, int, int);

int main(void)
{
    Tfunc *cadastro;
    int funcionarios, adicionais;
    printf("Quantos funcionarios? ");
    scanf("%d",&funcionarios);
    cadastro=inicializa(cadastro, funcionarios);
    if (!cadastro)
        return -1;
    printf("Quantos funcionarios a mais deseja adicionar? ");
    scanf("%d",&adicionais);
    cadastro = inclui_novos(cadastro, funcionarios, adicionais);
    if (!cadastro)
        return -1;
    printf("Lista de funcionarios cadastrados\n");
    imprime(cadastro, 0, funcionarios+adicionais);
    free(cadastro);
    return 0;
}
```

Desafio 3

- **(Baseado no problema “Margaridas”, OBI-2006)** Leopoldo é gerente de uma plantação de margaridas cuja produção abastece floriculturas em grandes cidades. As margaridas são plantadas em vasos dispostos em linhas e colunas, formando uma espécie de grade. Na plantação administrada por Leopoldo existem L linhas de vasos de margaridas, cada uma formada por C vasos. Para facilitar o gerenciamento, os vasos são organizados em lotes de M linhas e N colunas de vasos, sendo que **não existem sobreposições entre os lotes** (não existe nenhuma linha ou coluna comum a mais de um lote) e todos os lotes têm exatamente M linhas e N colunas. **A colheita é sempre feita em um único lote**, coletando-se todas as margaridas daquele lote que estejam prontas para a venda. Uma semana antes de fazer a colheita, os funcionários da plantação analisaram cada vaso e anotaram quantas margaridas estarão prontas para venda na semana seguinte. Leopoldo agora precisa da sua ajuda para determinar qual o **número máximo de margaridas que poderá ser colhido em um único lote de $M \times N$ vasos**.
-

Desafio 3 (cont.)

- Sua tarefa é escrever um programa que, dado um mapa da plantação contendo o número de margaridas prontas para venda em cada vaso, encontre qual o número máximo de margaridas que podem ser colhidas por Leopoldo (em um único lote).
 - **Entrada.** A entrada contém um único conjunto de testes que deve ser lido do dispositivo de entrada padrão. A primeira linha da entrada contém quatro números inteiros, **L**, **C**, **M** e **N**. **L** e **C** representam respectivamente o número de linhas ($L \geq 1$) e de colunas ($C \geq 1$) de vasos existentes na plantação. **M** e **N** representam respectivamente o número de linhas ($1 \leq M \leq L$) e de colunas ($1 \leq N \leq C$) dos lotes. As **L** linhas seguintes contêm **C** inteiros cada, representando o número de margaridas prontas para colheita no vaso localizado naquela linha e coluna. Note que L/M e C/N são sempre inteiros, pois não há linha ou coluna de vasos que pertença a mais de um lote.
 - **Saída.** Seu programa deve imprimir, na saída padrão, uma única linha que contém o número máximo de margaridas que podem ser colhidos em um lote de **M** x **N**.
-

Desafio 3 (cont.)

- Exemplos de entrada e saída:

Entrada 3 3 1 1 1 2 3 1 3 3 1 10 1 Saída 10	Entrada 4 4 2 1 1 2 3 4 5 6 7 8 1 10 5 2 1 5 9 10 Saída 15	Entrada 6 6 2 2 1 1 1 1 1 1 1 2 2 1 1 2 1 2 2 1 1 1 1 1 1 1 1 1 1 3 1 1 3 1 1 1 1 1 2 1 Saída 7
---	---	--

- Obs.:** Seu programa deverá utilizar uma matriz $L \times C$, **alocada dinamicamente**, para armazenar a quantidade de margaridas por vaso na plantação.
-